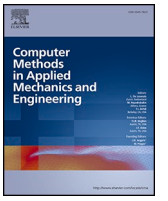
Computer Methods in Applied Mechanics and Engineering 426 (2024) 117000

Contents lists available at ScienceDirect

Computer Methods in Applied Mechanics and Engineering

journal homepage: www.elsevier.com/locate/cma



Anti-derivatives approximator for enhancing physics-informed neural networks

Jeongsu Lee

Department of Mechanical, Smart, and Industrial Engineering, Gachon University, Seongnam 13120, South Korea

ARTICLE INFO

Keywords:
Anti-derivatives approximator
Physics-informed neural networks
Adaptive activation
Piecewise linear approximation

ABSTRACT

This study presents a novel strategy for constructing an approximator for arbitrary univariate functions. The proposed approximation utilizes the anti-derivatives of a Fourier series expansion for the presumed piecewise function, resulting in a remarkable feature that enables the simultaneous approximation of an arbitrary function and its anti-derivatives. These anti-derivatives can be employed to discover solution curves for systems of ordinary differential equations based on an optimization scheme, even in the presence of chaotic dynamics. Additionally, the anti-derivatives approximator is extended as an adaptive activation function for physics-informed neural networks, leveraging the high-order differentiability of the anti-derivatives. Systematic experiments have demonstrated the outstanding merits of the proposed anti-derivatives-based approximator, including its ability to construct regression models for volatile data and their anti-derivatives, solve differential equations, and enhance the capabilities of physics-informed neural networks.

1. Introduction

Physics-Informed Neural Networks (PINNs) provide a sophisticated approach for representing the solution curve of governing equations through the framework of deep neural networks [1–3]. These solution curves, articulated by deep neural networks, are constrained to comply with their respective governing equations and initial and boundary conditions (ICs and BCs). This compliance is ensured by training deep neural networks to minimize the loss function that integrates the residuals of the governing equations and the soft constraints of ICs and BCs. The role of automatic differentiation within these deep neural networks is pivotal in calculating loss function, enabling the expression of differential equations and constraints on differentiated variables [1]. In this context, constraints are specifically tailored for points sampled within the problem domain, for example, space and time. Consequently, PINNs essentially represent a mapping function from spatial-temporal input variables to desired physical quantity, denoted as $\mathbf{y}(\mathbf{x}, t|\theta)$, where $\mathbf{y}$ represents output variables, $\mathbf{x}$ signifies spatial coordinates, t denotes time and θ encompasses trainable parameters within the neural networks.

The versatility of PINNs is evident in their ability to address a wide array of computational challenges, including formulating solution curves for forward problems [4–6], deciphering parameters in inverse problems to realize specific configurations [7–9], and assimilating experimental data to refine solution representations [10,11-12]. This broad-spectrum applicability has garnered intense interest from the scientific community, prompting a surge in investigative studies across diverse disciplines such as fluid mechanics [5,8,13], thermal engineering [4,14,15], structural analysis [16,17], and dynamic systems [18–20]. Moreover, the field has seen the advent of innovative training strategies for PINNs, specifically designed to enhance their accuracy and convergence rates. Among these

E-mail address: leejeongsu@gachon.ac.kr.

https://doi.org/10.1016/j.cma.2024.117000
Received 12 January 2024; Received in revised form 15 March 2024; Accepted 15 April 2024
Available online 24 April 2024

advanced methods, the amplification of gradients via the employment of higher-order differentials in governing equations [21], spatial-temporal decomposition strategies [22,23], and the adaptive refinement of training strategy [24,25] are particularly noteworthy.

In the realm of neural network architecture for PINNs, the Fully Connected Neural Network (FCNN) with tanh activation functions stands out as the most commonly used design [1–3]. PINNs typically consider an input dimension of less than $\mathbb{R}^4$, as they often address problems in physics and mechanics [13–20] characterized by governing equations expressed in time (t) and three-dimensional space (x, y, z). Therefore, elementary architectures, such as FCNNs, generally demonstrate successful capabilities in expressing the solution curves of given problems [2,3]. Specific issues that require a hierarchical data structure with connections between spatial-temporal nodes, for example, problems in multi-body dynamic systems [20], hyperelastic materials [26], and geological modeling [27], have seen successful implementations using Graph Neural Networks (GNNs). Additionally, extensions of GNNs that incorporate convolutional filters, known as Graph Convolutional Neural Networks (GCNNs), have been effectively employed to resolve complex problems such as convective heat and mass transfer [28,29]. These networks leverage graph data that maintain a hierarchy, connecting each node and representing the physical properties of the respective nodes.

One of the notable challenges in effectively implementing PINN is the optimal selection and utilization of activation functions. Activation functions are crucial as they enable neural networks to establish non-linear input-output relations. However, the non-linear profiles within some activation functions can lead to vanishing gradient problems, which severely hinder the training process of neural networks [30,31]. Functions with distinct non-zero slopes, particularly in their active regions, can significantly enhance gradient propagation through deep layers [32]. This has led to the widespread adoption of activation functions like the Rectified Linear Unit (ReLU) and its innovative variants, such as the Exponential Linear Unit (ELU) and the Gaussian Error Linear Unit (GELU), which feature small modifications in their transition ranges [33]. These functions are praised for maintaining healthier gradient flows across layers, thereby improving the training stability and model expressiveness of neural networks.

An intriguing conundrum arises when applying these advanced activation functions to PINN. In general, PINN are designed to construct the solution curve of differential equations, necessitating that this curve be differentiable to at least the leading order of the equations. However, activation functions like ReLU, ELU, and GELU, which converge to zero upon multiple differentiations due to their linear range for ensuring distinct non-zero slopes, pose a challenge. Despite their proven efficacy in conventional neural networks, PINNs struggle to fully leverage these types of activation functions to mitigate vanishing gradients. Therefore, the alternative use of activation functions like tanh or harmonic functions [34,35] has become a more popular strategy for constructing PINNs. However, these activation functions could potentially limit the expressive capability of PINNs compared to conventional neural networks.

As one approach to circumvent this problem, recent studies have actively employed adaptive activations, which incorporate trainable parameters into the activation function [34]. These trainable parameters in the activation function have been shown to prevent convergence to suboptimal points by providing additional scaling of internal variables [34,36]. Subsequently, the concept of Kronecker neural networks was introduced, incorporating trainable scaling parameters for both weights and biases in the linear mapping layer of neural networks [37]. In general, adaptive activation with trigonometric functions or the hyperbolic tangent function exhibits state-of-the-art capability in constructing PINN [34–37]. Despite the outstanding capabilities of adaptive activation, the functional profiles of original activation functions, such as sine or tanh, still limit their ability in preserving gradient flows compared to more recent innovative activation functions. Consequently, it remains a challenging problem in constructing PINN to find an appropriate activation profile that satisfies both requirements: 1) enabling multiple differentiations corresponding to the order of given differential equations, and 2) preserving gradient flows comparable to ReLU and its variants.

In this regard, finding an optimal strategy to configure the PINN remains an open problem. The limitations of PINN become more pronounced for certain extreme problems in mechanics and applied physics. For example, the issue of turbulence in fluid flows under high Reynolds number conditions remains an unresolved problem in PINN research [13]. Most fluid dynamic problems considered in PINN studies correspond to conditions with Reynolds numbers below 1000 [38,39]. Additionally, dynamic systems exhibiting chaotic behavior represent another challenging area for PINN [2]. Efforts to address these issues and enhance PINN performance have been actively pursued, including strategies such as balancing the loss components with additional regularization [24,40], employing domain decomposition to simplify the problems [22-23,25], incorporating conventional techniques from numerical analysis, for example, pseudo-viscosity [41,42], and employing alternative basis functions designed for specific target problems [43].

This study aims to complement recent extensive works on PINN by addressing the limitations of activation functions. It explores a novel strategy to construct a univariate approximation curve that can be applied to the activation functions of neural networks. The proposed univariate approximator draws inspiration from the well-known advantages of the ReLU function, which exhibits a piecewise step function at first-order differentiation and converges to zero at second-order differentiation. This characteristic enables the ReLU activation to effectively search for optimal piecewise linear approximations, thereby circumventing vanishing gradient issues [30,31]. To confer the same benefits, the proposed approximator starts with a piecewise constant function and constructs the approximation curve by integrating this function. The formulation of the integrated function is realized through the Fourier orthogonal expansion of the piecewise constant function. Consequently, the approximator is termed ADA-F (Anti-Derivatives Approximator from Fourier series expansion) "Anti-derivatives" refers to the indefinite integrations from the piecewise constant function, with piecewise slopes and integration constants determined during training. By adjusting the order of integration, the proposed univariate approximation could provide a nonlinear mapping that exhibits a constant function at the desired order of differentiation, which then converges to zero at one order higher.

In the following, the notable benefits of ADA-F have been demonstrated through systematic experiments, both in its standalone application and when integrated into PINN. When used independently, ADA-F has shown the capability to precisely express arbitrary

target functions with high volatility, utilizing a limited number of trainable variables. A distinctive feature of ADA-F is its ability to simultaneously capture the anti-derivatives of the solution curve it identifies. This remarkable attribute enables ADA-F to accurately model solution curves of Ordinary Differential Equations (ODEs), even in scenarios involving chaotic dynamics. Furthermore, integrating ADA-F into PINNs has significantly enhanced their capacity to effectively tackle a wide array of benchmark problems related to Partial Differential Equations (PDEs). These include the Burger's equation, the static Euler beam problem, and the Kovasznay flow. The incorporation of ADA-F in these cases has not only improved the accuracy of the solutions but also expanded the range of problems that PINNs can effectively address.

2. Proposition of anti-derivative-based approximation

Revisiting the Universal Approximation Theorem [44], an arbitrary compactly-supported continuous function can be represented by a combination of the ramp function $r(x)$, defined as –1 for $x < -1$, x for $|x| \leq 1$, and 1 for $x > 1$, respectively. Neural networks incorporating a properly designed activation function can realize this combination of ramp functions, leading to the piecewise-linear or constant approximation of a target function. The challenge, however, is that gradients often vanish before the neural networks can achieve the ideal solution during gradient descent steps. To address this issue, the present study proposes a reverse process termed ADA-F, which aims to optimize the neural networks by using a mathematical expression for the presumed optimal piecewise-linear approximation, as schematically illustrated in Fig. 1.

Suppose that neural networks $g(x|\ \theta)$ are constructed from trainable variables θ, attempting to approximate an arbitrary mapping function $f : x \in \mathbb{R} \rightarrow y \in \mathbb{R}$, expressed as $y = f(x)$, and defined in the range of $x \in [x_{\min},\ x_{\max}]$. The function f is assumed to be at least piecewise continuous within the defined interval. The optimal approximator within an arbitrarily small tolerance can be expressed as a piecewise-linear or constant approximation, denoted as $g(x|\ y_0,\ \lambda_i, x_i)$, where y_0 corresponds to $f(x_{\min})$, λ_i and x_i represent the slope and endpoint of the i th piece, respectively, with a properly given number of pieces N_p. The derivative of $g(x)$ with respect to x is then expressed as a piecewise constant function, whose value corresponds to λ_i in the range of $[x_{i-1},\ x_i]$:

$$\frac{dg(x)}{dx} = \lambda_i \qquad \text{for } x_{i-1} < x < x_i \tag{1}$$

Orthogonal expansion provides a convenient mathematical expression to handle the piecewise constant function, which can be represented by a convergent series:

$$\frac{dg(x)}{dx} = \sum_{m=0}^{\infty} a_m \phi_m(x) \tag{2}$$

where $\phi_m(x)$ denotes orthogonal functions and a_m corresponds Fourier constants of the orthogonal series [45]. Then, $g(x)$ can be

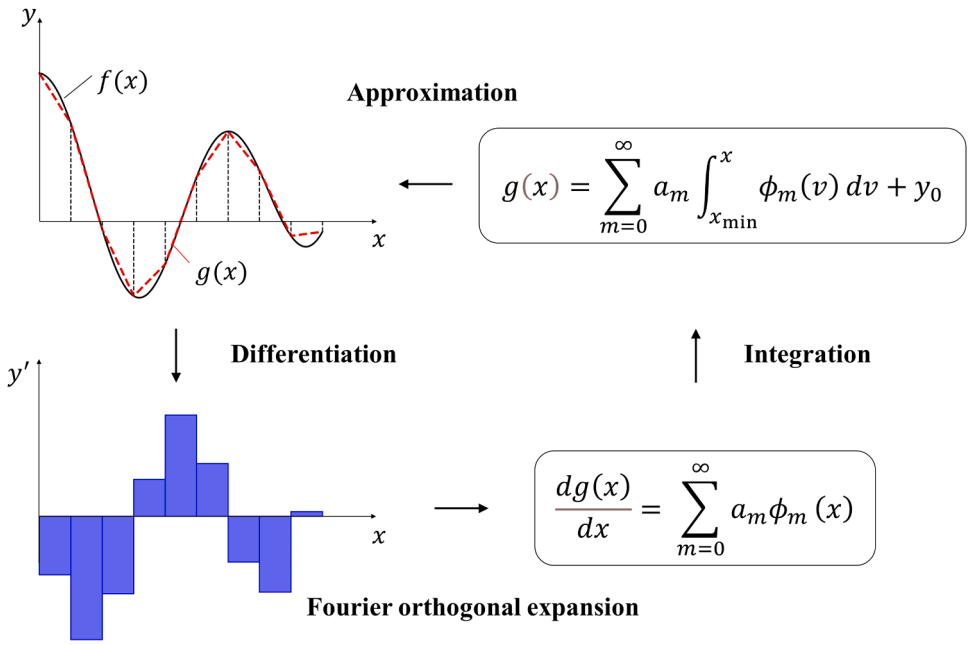


Fig. 1. Schematic illustration of ADA-F. The target function $f(x)$ is approximated using the piecewise-linear function $g(x)$. Leveraging the fact that the derivative of a piecewise-linear function is a piecewise step function, the approximation is formulated based on the integration of the Fourier orthogonal expansion of the piecewise constant function.

formulated from the first anti-derivative of the adopted orthogonal function as a definite integral:

$$g(x) = \sum_{m=0}^{\infty} a_m \int_{x_{\min}}^{x} \phi_m(v)dv + y_0 \tag{3}$$

2.1. Employing Fourier series expansion

One simple choice of the orthogonal functions could be the Fourier series with a half-range expansion considered in the range of $[0, L]$:

$$\frac{d\widetilde{g}(x^*)}{dx^*} = \sum_{m=0}^{\infty} \left(\alpha A_m \cos\frac{m\pi x^*}{L} + \beta B_m \sin\frac{m\pi x^*}{L} \right) \tag{4}$$

where x^* corresponds to the linear transformation of the variable x from the range of $[x_{\min},\ x_{\max}]$ to $[0, \gamma L]$. It is then expressed as $x^* = \mathscr{L}(x) = Wx + b$ with a scalar weight W and a scalar bias b. The γ controls the scaling of linear transformation and defined in the range of $(0,\ 1]$. It is noted that the mapping control factor γ does not necessarily equal one. The α and β adjust the balance between even and odd functions in the approximator, obeying the constraint, $\alpha + \beta = 1$. Fourier coefficients A_m and B_m for $m > 1$ are then represented with λ_i^* and x_i^*:

$$\begin{aligned} A_m &= \frac{2}{L}\int_0^L \frac{d\widetilde{g}(x^*)}{dx^*}\cos\frac{m\pi x^*}{L}dx^* = \frac{2}{m\pi}\sum_{i=1}^{N_p}\lambda_i^*\left(\sin\frac{m\pi x_i^*}{L} - \sin\frac{m\pi x_{i-1}^*}{L}\right) \\ B_m &= \frac{2}{L}\int_0^L \frac{d\widetilde{g}(x^*)}{dx^*}\sin\frac{m\pi x^*}{L}dx^* = -\frac{2}{m\pi}\sum_{i=1}^{N_p}\lambda_i^*\left(\cos\frac{m\pi x_i^*}{L} - \cos\frac{m\pi x_{i-1}^*}{L}\right) \end{aligned} \tag{5}$$

where $x_i^* = \mathscr{L}(x_i)$ and $A_0 = \int_0^L \frac{d\widetilde{g}(x^*)}{dx^*}dx^*$. Finally, the $\widetilde{g}(x^*)$, which attempts to approximate the $f(\mathscr{L}^{-1}(x^*))$, can be expressed by integrating (10) as:

$$\widetilde{g}(x^*) = \sum_{m=1}^{\infty}\left[\alpha\frac{LA_m}{m\pi}\sin\frac{m\pi x^*}{L} - (1-\alpha)\frac{LB_m}{m\pi}\left(\cos\frac{m\pi x^*}{L} - 1\right)\right] + \alpha A_0 x^* + y_0 \tag{6}$$

It is noted that this formulation retains the elementary operations of neural networks expressed as a scalar quantity: $\sigma \circ (Wx + b)$, where W represents the weight, b signifies the bias, x denotes the input signals, and σ correspond to the activation function.

The optimization problem can be proposed to determine the formulation (6) by adopting hyperparameters as follows. First, the mapping control factor γ, the scale of half-range expansion L and the number of linear pieces N_p are determined as hyperparameters. Second, the number of orthogonal modes N_m is introduced to express a truncated series solution as

$$\widetilde{g}(x^*) = \sum_{m=1}^{N_m}\left[\alpha\frac{LA_m}{m\pi}\sin\frac{m\pi x^*}{L} - (1-\alpha)\frac{LB_m}{m\pi}\left(\cos\frac{m\pi x^*}{L} - 1\right)\right] + \alpha A_0 x^* + y_0 \tag{6a}$$

Last, the distribution of linear pieces x_i^* is adopted from fixed values, for example, uniform distribution over the input range. It is noted that the x_i^* can be determined from the optimization by introducing additional constraints describing x_i^*, which is presented in *Appendix*. However, it is a well-functioning strategy to resolve the target problem with fixed values of x_i^* reducing the computational cost without compromising accuracy. Therefore, the optimization problem to find optimal y_0, λ_i^*, and α can be given, for example:

$$\underset{y_0,\ \lambda_i^*, \alpha}{\operatorname{argmin}} \sum \left(f\left(\mathscr{L}^{-1}(x^*)\right) - \widetilde{g}(x^*)\right)^2 \tag{7}$$

2.2. Extension to anti-derivatives

The central merit of the proposed ADA-F is that the formulation (6) can be integrated multiple times to describe anti-derivatives of a target function. The formulation of ADA-F and its anti-derivatives are then expressed as

$$\mathrm{ADAF}^{N+1}(x^*) = \int_0^{x^*} \mathrm{ADAF}^N(v)dv + y_{N+1} \mathrm{for} N \geq 0, \tag{8}$$

$$\mathrm{ADAF}^0(x^*) = \sum_{m=1}^{N_m}\left[\alpha\frac{LA_m}{m\pi}\sin\frac{m\pi x^*}{L} - (1-\alpha)\frac{LB_m}{m\pi}\left(\cos\frac{m\pi x^*}{L} - 1\right)\right] + \alpha A_0 x^* + y_0$$

The exact mathematical formulations up to the 3rd-order ADA-F are presented in the *Appendix*. Higher-order ADA-Fs can be

adopted to tackle problems involving differential equations based on neural networks, which require high-order derivatives to express differential equations. Furthermore, ADA-F can be incorporated with deep neural networks as a novel layer possessing adaptive activation, a development that is shown to enhance the expressive capability of neural networks.

3. Standalone implementation of ADA-F

The analysis begins by examining the capability of the standalone implementation of ADA-F. The ADA-F constructs a univariate mapping function from the input variable x to the output variable y, which can be adopted to resolve a regression problem for a univariate relation. Subsequently, the study extends the capability of ADA-F to uncover the anti-derivatives of a regressed relation in the context of a system of ordinary differential equations.

3.1. Regression on a volatile function and automatically found anti-derivatives

This example illustrates the capability of ADA-F to find a regression curve for univariate measurement data, while simultaneously capturing the anti-derivatives of the regressed relation, as schematically illustrated in Fig. 2(a). The regression on a volatile function based on uncertain measurements is considered, which corresponds to uncovering the trend described by:

$$y = \sin(x) + \sin(2x) + 0.5\cos(6x) + \cos\left(x^2\right) + \sqrt{x+5} \tag{9}$$

in the range of $x \in [-5,\ 5]$. The target function is arbitrarily designed to resemble white noise, as shown by the black-dotted line in Fig. 2(b). Consequently, capturing the relationship between the input and output variables becomes a troublesome task for regression algorithms that employ explicit mathematical models. The measurements from equally distributed points, including a random error from the Gaussian distribution $\mathcal{N}(0,\ 0.2^2)$, are given for training, which are depicted as blue dots in Figure 2(b).

The zeroth-order ADA-F is constructed based on the formulation (8) with hyperparameters: the number of orthogonal modes $N_m = 40$, the number of linear pieces $N_p = 40$, the mapped scale of half-range expansion $L = 1$, the mapping control factor $\gamma = 1$, and a fixed balancing factor between even and odd functions $\alpha = 0.5$. The locations of pieces x_i^* are equally distributed in the $[0,\ 1]$ interval as fixed constants. Thereby, the ADA-F has 41 trainable variables, comprising 40 slopes in linear pieces λ_i and the constant of integration y_0. The L-BFGS-B algorithm [46] is adopted for resolving the optimization problem (7) via the Scipy implementation [47].

The solution curve described ADA-F is plotted as a red line in Figure 2(b). As baseline methods, the predictions of conventional regression models—including SVR (support vector regression), GPR (Gaussian process regression), and BRR (Bayesian ridge regression)—are depicted as well, with green, grey, and yellow lines, respectively. Whereas the conventional regression models oversimplify the trend of the volatile function, the ADA-F demonstrates remarkable expressive capability. More surprisingly, the ADA-F can easily capture the anti-derivatives of the target function. Figure 2(c) compares the automatically found anti-derivatives (red, blue, and green lines) with analytic solutions (black-dotted lines), where the integration constants are determined as the anti-derivatives being zero at $x = -5$. It is important to note that this example is not intended to provide solid evidence convincing that the ADA-F is a superior regression model compared to conventional models. However, the experiment demonstrates the remarkable capability of the ADA-F in expressing complex input-output relations and even in capturing its anti-derivatives.

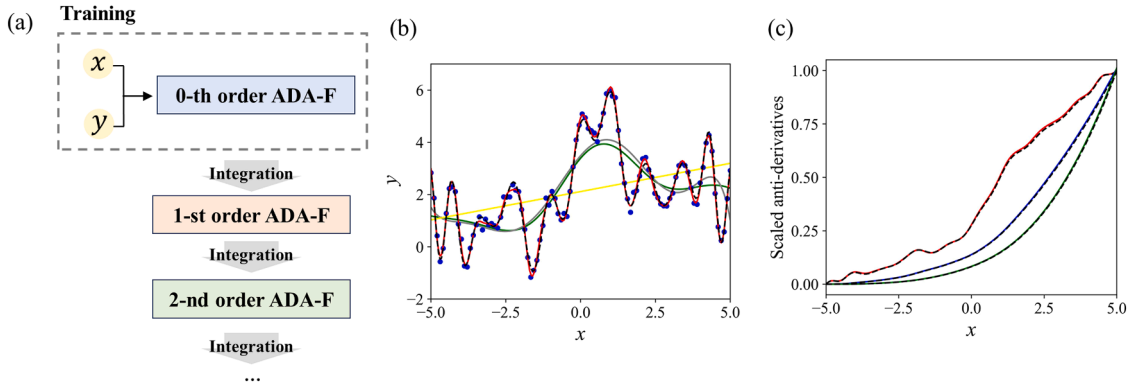


Fig. 2. Standalone implementation of ADA-F. (a) Schematic illustration of ADA-F for regression of univariate data and simultaneous identification of its anti-derivatives. (b) Regression experiments on a volatile function. Comparison of ADA-F (red line) with conventional regression models such as SVR (green), GPR (grey), and BRR (yellow), where the sampled data points from the analytic solution (black-dotted line) are depicted as blue dots. (c) Automatically found anti-derivatives from ADA-F. The red, blue, and green lines represent the scaled anti-derivatives of the target function, iteratively discovered by increasing the order of integration and constraining the anti-derivatives to be zero at $x = -5$. Black-dotted lines correspond to the analytic solutions.

3.2. Solving chaotic dynamics of the Lorenz system

The chaotic dynamics of the Lorenz system are considered based on the following system of differential equations:

$$\left(\frac{dx}{dt}, \frac{dy}{dt}, \frac{dz}{dt}\right) = (\sigma(y-x), x(\rho-z)-y, xy-\beta z) \tag{10}$$

The solution curve of the Lorenz system can be easily obtained by current numerical schemes, which involve iterative time advancements starting from hard constraints on the initial conditions. However, obtaining a solution curve of the Lorenz system in terms of an optimization problem presents a challenge due to its chaotic behavior, wherein small changes in the initial state can lead to dramatic changes across the entire solution. For instance, the PINN solutions for the Lorenz system as a forward problem are available only for a few implementations [48,49], with limited accuracy [48] and reproducibility [49]. Resolving chaotic differential equations is one of the representative limitations of current state-of-the-art PINN methodologies [2].

The solution curve of the Lorenz equation is constructed based on ADA-F as schematically illustrated in Fig. 3(a). First, the first-order differentials, $\frac{dx}{dt}$, $\frac{dy}{dt}$ and $\frac{dz}{dt}$, are interpreted with zeroth-order ADA-Fs, with the input time scale mapping as $t^* = \mathscr{L}(t) = Wt$, resulting in the expression $\left(\frac{dx}{dt}, \frac{dy}{dt}, \frac{dz}{dt}\right) = W(\mathrm{ADAF}_x^0(t^*), \mathrm{ADAF}_y^0(t^*), \mathrm{ADAF}_z^0(t^*))$. The zeroth-order ADA-F describes the differentiation in mapped input t^*. Then, the constant factor in the right-hand side is adopted from $\frac{dt^*}{dt} = W$. This allows the expression of solution curves for x, y and z, with the first-order ADA-Fs as ADAF_x^1, ADAF_y^1 and ADAF_z^1, respectively. Therefore, the Lorenz Eq. (10) can be rewritten with a set of ADA-Fs as

$$W \cdot \mathrm{ADAF}_x^0 = \sigma\left(\mathrm{ADAF}_y^1 - \mathrm{ADAF}_x^1\right) \tag{10a}$$

$$W \cdot \mathrm{ADAF}_y^0 = \mathrm{ADAF}_x^1\left(\rho - \mathrm{ADAF}_z^1\right) - \mathrm{ADAF}_y^1$$

$$W \cdot \mathrm{ADAF}_z^0 = \mathrm{ADAF}_x^1 \mathrm{ADAF}_y^1 - \beta \mathrm{ADAF}_z^1$$

The residuals of the governing equations (10′) are employed as a loss function for the optimization problem, which enforces that the

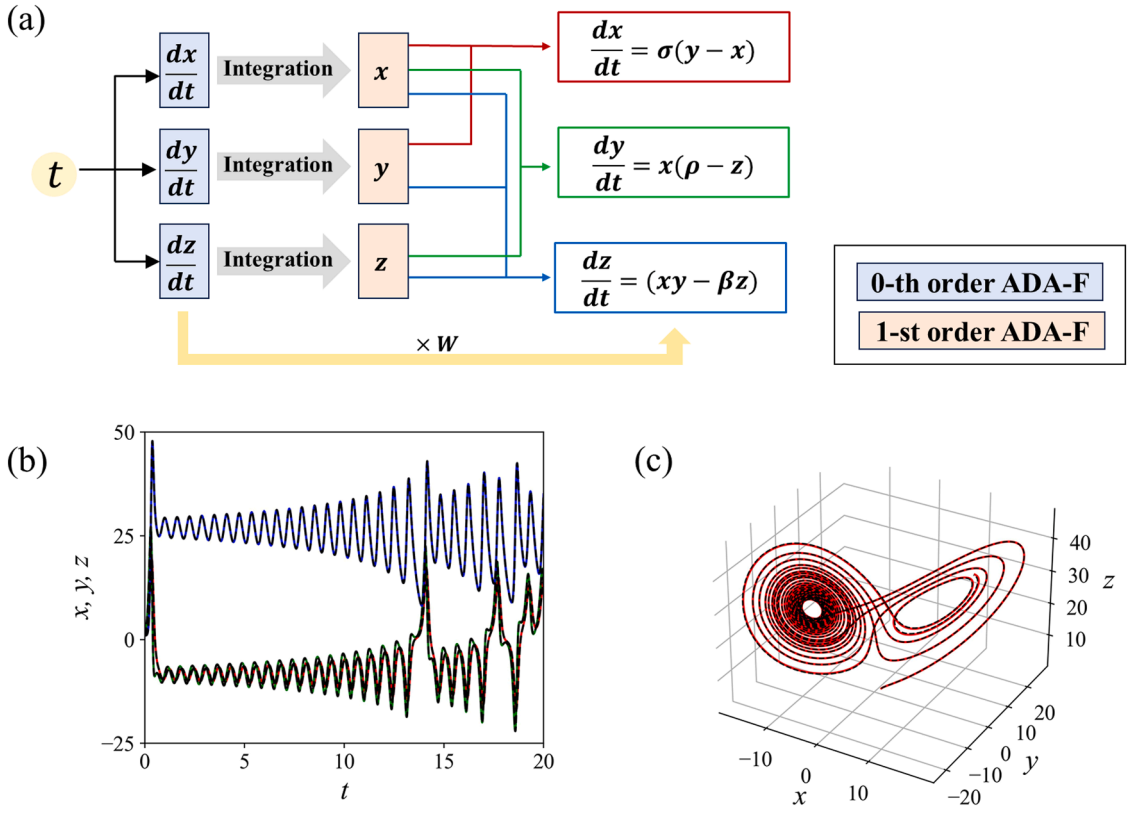


Fig. 3. Chaotic dynamics of the Lorenz system. (a) Schematic illustration of ADA-Fs configuration for the ODE residuals of the Lorenz system. (b) The time evolution of x, y, and z are depicted as red, green, and blue lines, respectively, with corresponding numerical solutions (black-dotted lines). (c) Lorenz attractor depicted from ADA-F (red line) and the numerical solution (black-dotted line).

solution of ADA-Fs obeys the Lorenz equation. Initial conditions are employed as hard constraints by fixing the integration constants. The ADA-Fs are constructed with hyperparameters: the number of orthogonal modes $N_m = 10$, the number of linear pieces $N_p = 10$, the mapped scale of half-range expansion $L = 1$ and the mapping control factor $\gamma = 0.2$. The time-marching strategy proposed in several PINN research [50] is employed, which iteratively trains independent models with a local time step τ until the desired time span is reached. The local time step τ is selected as 0.1, leading to 200 repetitive trainings of ADA-Fs until the desired time horizon of 20 is reached. The linear mapping is then expressed as $t^* = Wt = (\frac{\gamma L}{\tau})t$. The training variables in ADA-Fs are optimized by the L-BFGS-B algorithm [46,47].

In this experiment, the solution curves are obtained under the conditions of $\sigma = 10.$, $\rho = 28.0$, and $\beta = 8.0/3.0$ for the time span of $[0,\ 20]$, which corresponds to the condition considered in [49]. Fig. 3(c and d) exhibits the solution curves predicted by ADA-F (red, blue, and green lines for x, y and z, respectively) in comparison with the solution of the numerical scheme (black dotted lines). The solution curve of the ADA-F shows remarkable agreement with the numerical solution, exhibiting $L2$ relative errors of 7.986×10^{-3}, 1.162×10^{-2}, and 4.932×10^{-2} for x, y and z, respectively. Even though these results are obtained from the standalone implementation of the ADA-F, they correspond to state-of-the-art accuracy compared to the PINN solutions [49]. These two examples illustrate that ADA-F is a versatile approximator for arbitrary univariate relations. However, the standalone implementation of ADA-F is hard to expand to multivariate relations. Therefore, integration with deep neural networks is essential for more general usage, as demonstrated in the following, which presents major advantages of ADA-F.

4. Implementations with PINNs

Suppose that ADA-F aims to capture functional mapping $f : x_{ij} \in \mathbb{R}^{p\times q} \rightarrow y_{ij} \in \mathbb{R}^{p\times q}$ with arbitrary input sizes p and q, an explicit ADA-F layer is proposed as

$$x^*_{ij} = \mathscr{L}(x_{ij})\ y_{ij} = \mathrm{ADAF}^N_{ij}\left(x^*_{ij}\right) \tag{11}$$

where N is the order of ADA-F layer given as a hyperparameter, $\mathscr{L}$ corresponds to the linear operator implying the input scale mapping. Although the formular (11) provides explicit implementation of ADA-F into neural networks, it dramatically increases computational cost by input dimensions as it constructs independent ADA-F for every single input variable. As a remedy, a simplified implementation of ADA-F layer can be suggested as

$$y_{ij} = \mathrm{ADAF}^N \circ x^*_{ij} \tag{11a}$$

In this formular, a single ADA-F is constructed as an elementwise operation for input variables like the activation function. Therefore, the simplified ADA-F layer can be interpreted as one of adaptive activation whose profile is determined by optimization [51,52].

Furthermore, the linear operator is expressed as $\mathscr{L}(x_{ij}) = W_{ij} \cdot x_{ij} + b_{ij}$ with $W_{ij}, b_{ij} \in \mathbb{R}^{p\times q}$. The variables W_{ij} and b_{ij} can be found from the training or given as fixed constants in the configuration. The input can be extended to arbitrary dimension.

Upon testing the hyperparameters appended in Appendix E, the configuration showing stable performance using a reduced number of trainable parameters is proposed as follows: 1) Employing only the Fourier cosine expansion is sufficient to enhance the capability of neural networks, corresponding to $\alpha = 1.0$; 2) Maintaining the same number of linear pieces and bases usually exhibits better performance, which corresponds to $N_p = N_m$; 3) The linear operator $\mathscr{L}$ can be replaced by a Dense layer; 4) Introducing a linear scaling after the ADA-F layer helps in enhancing convergence by regulating the scale of the output; 5) Employing three or five truncated modes suffices when the ADA-F layer is used right before the prediction layer; 6) The ADA-F layer can be placed at the top of the neural networks following the linear Dense layer for input variables, particularly for extreme problems such as high Reynolds number flows, where a large number of truncated modes are favorable, for example, N_m>15.

Three benchmark problems in PINN are investigated as follows: 1) Burger's equation, 2) static Euler beam problem, and 3) Kovasznay flow. The Burger's equation corresponds to a 1-dimensional non-linear differential equation describing convection-diffusion phenomena, which is widely used as a benchmark problem in PINN [1,21]. The Euler beam problem regards a fourth-order differential equation. Thereby, this problem can test the capability of the ADA-F layer to resolve a high-order differential equation. Lastly, the Kovasznay flow is governed by a system of partial differential equations, such as continuity and incompressible Navier-Stokes equations.

4.1. Burger's equation

The capability of the ADA-F extension has been examined for resolving the 1D Burger's equation, describing nonlinear convection-diffusion phenomena based on the 2nd-order PDE:

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} = \upsilon\frac{\partial^2 u}{\partial x^2} \tag{12}$$

The temporal evolution of the initial sinusoidal wave profile, $u(x,0) = -\sin(\pi x)$, with periodic boundary conditions in domain $x \in [-1,\ 1]$ and kinematic viscosity $\upsilon = \frac{0.01}{\pi}$, is considered. The explicit solution of Burger's equation can be described based on the hyperbolic tangent function in the reference frame moving with the traveling wave [53]. Therefore, the PINN incorporating the tanh

activation is shown to easily obtain a very accurate solution curve of Burger's equation, corresponding to an order of 10^{-4} in terms of the $L2$ relative error [1].

The capability of the ADA-F extension has been evaluated by varying the architectures of Fully Connected Neural Networks (FCNN), as presented in Table 1. As a proposed elaboration, the 2nd-order ADA-F is implemented after the last hidden dense layer, replacing the tanh activation. The outcome of ADA-F is delivered to the tanh function for leveraging the shape of the tanh function to constitute the traveling wave solution. The final prediction is made by linear superposition to produce the solution curve of $u(x,t)$. The first-order ADA-F layer is adopted with the following hyperparameters: the number of orthogonal modes $N_m = 3$, the number of linear pieces $N_P = 3$, the mapped scale of half-range expansion $L = 1$ and the mapping control factor $\gamma = 1.0$, with a simplified formular (11′). The balancing factor (α) and integration constants (y_0 and y_1) are fixed to one and zeros to reduce computational cost. It is noted that the ADA-F extension introduces only 4 additional trainable parameters compared to the baseline PINN.

The schematic illustration of the ADA-F extension is presented in Fig. 4(a). The collocation points are randomly selected in the domain, with 200, 200, and 10,000 samples for initial, boundary, and residual points, respectively. The training starts with the Adam optimizer [54] at a learning rate of 1×10^{-2} for 200 epochs. After that, the optimizer is switched to the L-BFGS-B algorithm based on the Scipy implementation and is maintained until convergence [46,47]. This optimizer configuration was determined based on preliminary experiments detailed in Appendix C, where Adam [54], AdamW [55], and L-BFGS-B [46] were compared. The selected optimizer settings are applied to every PINN model across benchmark problems.

Figures 5(b) and (c) compare the performance of the baseline PINN (blue solid line) and the ADA-F extension for PINN (red solid line) applied to Burger's equation, focusing on training loss and $L2$ relative error. Given the variability in convergence and accuracy of the PINN solution across trials, results from 10 random experiments are presented. The predicted solution of the ADA-F extension is showcased in Figures 5(d) and (e), depicting the temporal evolution of the predicted $u(x,t)$ and its contour plot in the space-time domain, respectively. Figure 5(d) illustrates the comparison between the predicted solution (solid line) and the exact solution (dashed line) over varying times from 0.0 to 0.99, represented in a gradient from dark to light colors. The contour plot in Figure 5(e) demonstrates that the presented ADA-F extension successfully captures the appearance of a shock wave at $x = 0$. Overall, the experiments demonstrate that the ADA-F extension can achieve order-of-magnitude improvements in both training loss and accuracy compared to the baseline PINN.

The additional computational costs associated with the ADA-F extension have been analyzed and are summarized in Table 2. Implementing the ADA-F layer incurs significant computational costs due to the increased complexity in calculating gradient flow, which results in approximately three to five times more computational time compared to identical neural network architectures. However, the ADA-F extension can achieve accurate solutions with a considerably reduced number of trainable variables in the neural networks. For example, an ADA-F extension with a neural network architecture of 4 hidden layers, each with 20 neurons, can exhibit a four-digit decimal $L2$ relative error, corresponding to 6.787×10^{-4}, with just 1345 trainable parameters. In contrast, the baseline PINN could achieve comparable capabilities with an architecture of 4 hidden layers of 60 neurons each, corresponding to 11,221 trainable parameters. Therefore, the ADA-F extension can reduce the size of the neural networks by approximately ten times, leading to significantly reduced computational efforts to obtain comparable accuracy in solutions. This is evidenced by the elapsed time comparison between the baseline PINN (4×60) and the ADA-F (4×20), where the results are 356 s for the ADA-F extension and about 1103 s for the baseline PINN.

4.2. Static Euler beam

The static cantilevered Euler beam under uniform load condition is described by

$$\frac{d^4u}{dx^4} + 1 = 0 \tag{13}$$

where $u(x)$ describes the transverse displacement of the cantilevered beam in coordinate $x \in [0,\ 1]$. The fixed and free end boundary conditions are employed at each boundary as $u(0) = 0$ and $u'(0) = 0$ and $u''(1) = 0$ and $u'''(1) = 0$, respectively. The baseline architecture is constructed with FCNN by varying number of hidden layers as 2 and 4, and number of neurons in each layer as 10 and 20. The 3rd-order ADA-F layer is employed because the governing equation shows that the 4th-order derivative corresponds to a constant

Table 1
Burger's Equation: Capability comparison of the baseline and ADA-F extension across varying neural network architectures. metrics evaluated include training loss, L1 absolute errors, and L2 relative errors. Text in bold black denotes the best-performing result. The top results are presented from ten trials, based on parallel computations.

Architectures		Baseline			ADA-F		
Layers	Neurons	Loss	$L1$ absolute	$L2$ relative	Loss	$L1$ absolute	$L2$ relative
2	20	1.879E-03	1.093E-02	4.137E-02	5.739E-05	1.465E-03	7.668E-03
2	40	5.402E-04	5.362E-03	2.541E-02	3.973E-05	8.246E-04	3.551E-03
2	60	3.226E-04	5.277E-03	2.893E-02	3.488E-05	8.517E-04	3.241E-03
4	20	5.256E-06	2.660E-04	7.640E-04	2.863E-06	1.774E-04	6.787E-04
4	40	3.252E-06	2.005E-04	9.005E-04	1.219E-06	9.232E-05	3.331E-04
4	60	5.873E-06	2.518E-04	6.269E-04	**6.118E-07**	**6.694E-05**	**1.934E-04**

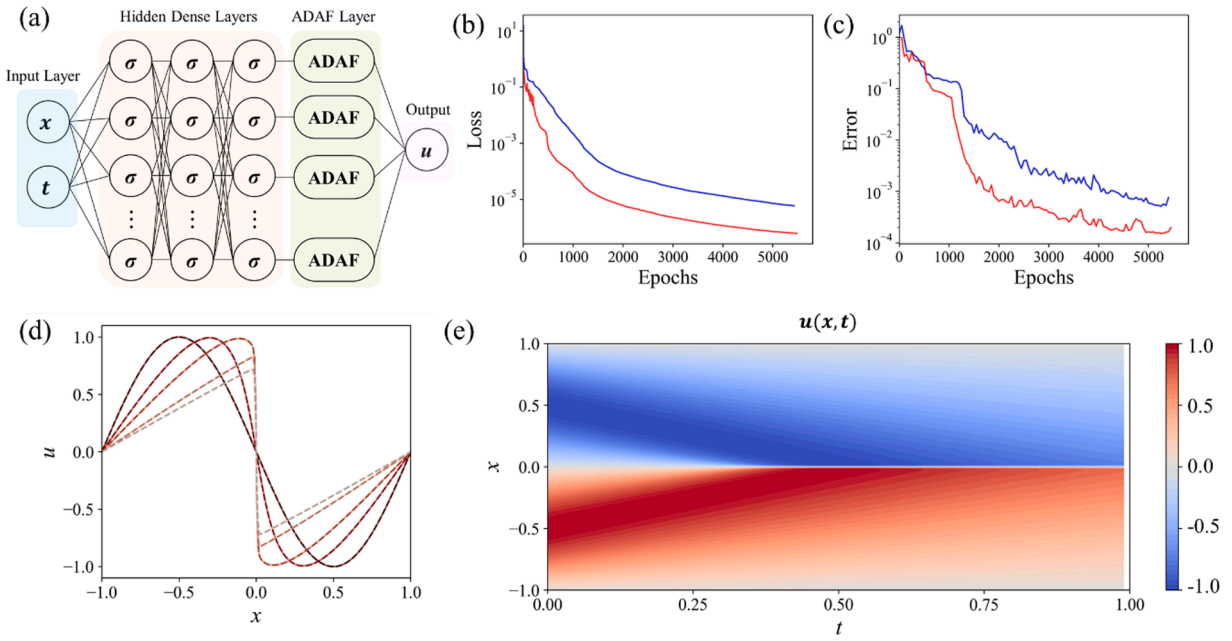


Fig. 4. Comparison of Vanilla PINN and ADA-F Extension for Burger's Equation. (a) Schematic illustration of the ADA-F extension. (b) Plots of training loss and (c) $L2$ relative error versus epochs for example cases with 4 hidden layers of 40 neurons each. The blue and red lines represent the results of the baseline model and the ADA-F extension, respectively. (d) Temporal evolution of predicted u versus x by the ADA-F extension (solid line) compared with the exact solutions (dashed line), where the temporal snapshots are depicted in a gradient from dark to light reds, representing chronological order. (e) Contour plot of the predicted $\boldsymbol{u}$ in the space-time domain.

Table 2
Burger's Equation: Comparison of convergence between the baseline and ADA-F extension across varying neural network architectures. Metrics used include the number of iterations, CPU time in seconds per epoch, and total elapsed time. The convergence results are based on the best-performing instances, determined by training loss, from five repetitive runs. Text in bold black denotes the architecture that exhibits comparable accuracy between the baseline and ADA-F extension, serving as a representative comparison case.

Architectures		Baseline			ADA-F		
Layers	Neurons	No. of Iterations	Time/Epoch (*sec*/epoch)	Time (*sec*)	No. of Iterations	Time/Epoch (*sec*/epoch)	Time (*sec*)
2	20	11,121	0.0088	97.945	11,376	0.0336	382.490
2	40	7656	0.0409	313.298	8317	0.1730	1438.728
2	60	7824	0.0681	532.530	6076	0.2690	2902.905
4	20	6076	0.0194	117.691	**5977**	**0.0597**	**356.947**
4	40	5618	0.0897	503.952	5070	0.2460	1247.082
4	60	**7465**	**0.1478**	**1103.349**	4866	0.5204	2531.924

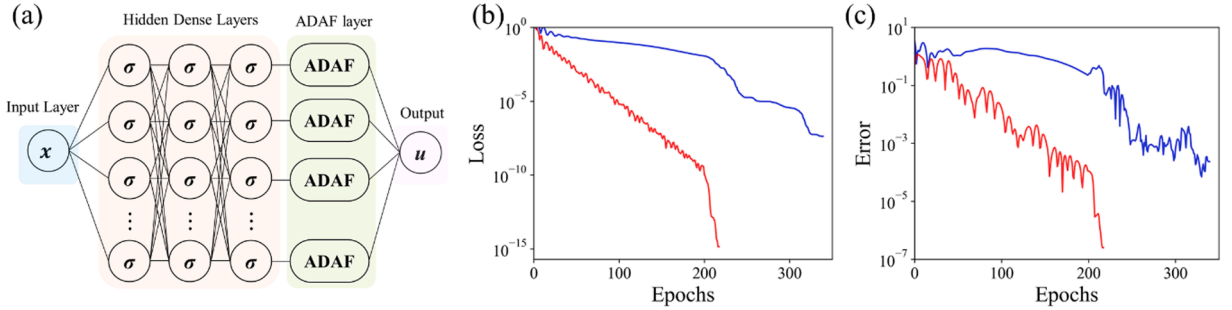


Fig. 5. Comparison of Vanilla PINN and ADA-F Extension for the Static Euler Beam Problem. (a) Schematic illustration of the ADA-F extension. (b) Training loss plots and (c) L2 relative error versus epochs obtained from neural networks with 2 hidden layers of 20 neurons each. The blue and red lines represent the results of the baseline model and the ADA-F extension, respectively.

function. The ADA-F extension is proposed to replace the tanh activation at the last hidden layer with an ADA-F layer of the scale of half-range expansion $L = 1$, the number of orthogonal modes $N_m = 3$ and the number of linear pieces $N_P = 3$ with the identity linear mapping. Thereby, the ADA-F extension require 7 additional training parameters, which is negligible compared to the total size of the neural networks. The collocation points are adopted from 20 random samples in the x domain.

The PINN architecture incorporating the ADA-F layer is schematically illustrated in Fig. 5(a). The performance of the vanilla PINN and the ADA-F extension for the architecture having 2 hidden layers and 20 neurons each is compared in Figs. 5(b) and (c) in terms of training loss and accuracy, respectively. The entire experimental results are summarized in Table 3, where the capability of the ADA-F extension is examined in terms of training loss, L1 and L2 accuracies. The ADA-F extension presents dramatic improvement in static Euler beam bending problem, summarized as the 7 orders of magnitude improvement in training loss and 3 orders of magnitudes improvements in terms of $L1$ absolute and $L2$ relative errors. It is remarkable that the ADA-F extension can significantly improve the capability of neural networks only with 7 additional training parameters. The convergence of the ADA-F extension is further examined in Table 4. The additional CPU time required is significant, amounting to approximately a threefold increase in time per epoch. Despite these increased expenses, the ADA-F extension can dramatically reduce the number of iteration steps required for convergence. Therefore, the ADA-F extension can achieve significantly better solutions compared to the baseline PINN, with comparable total computing times.

4.3. Kovasznay flow

The Kovasznay flow governed by continuity and 2D steady incompressible Navier-Stokes equations is examined as

$$\frac{\partial u}{\partial x}+\frac{\partial v}{\partial y}=0 \tag{14}$$

$$u\frac{\partial u}{\partial x}+v\frac{\partial u}{\partial y}=-\frac{\partial p}{\partial x}+\frac{1}{Re}\left(\frac{\partial^2 u}{\partial x^2}+\frac{\partial^2 u}{\partial y^2}\right)$$

$$u\frac{\partial v}{\partial x}+v\frac{\partial v}{\partial y}=-\frac{\partial p}{\partial y}+\frac{1}{Re}\left(\frac{\partial^2 v}{\partial x^2}+\frac{\partial^2 v}{\partial y^2}\right)$$

The governing equations describe the velocity fields, $u(x,y)$ and $v(x,y)$, and the pressure field $p(x,y)$, given the Reynolds number (Re). The exact solution of the Kovasznay flow has been found [56]:

$$u(x,y)=1-e^{\lambda x}\cos(2\pi y) \tag{15}$$

$$v(x,y)=\frac{\lambda}{2\pi}e^{\lambda x}\sin(2\pi x)$$

$$p(x,y)=\frac{1}{2}\left(1-e^{2\lambda x}\right)$$

The problem is considered in the domain of $x\in[-0.5,\ 1.0]$ and $y\in[-0.5,\ 1.5]$, with a Reynolds number of 40. The Dirichlet boundary conditions are imposed based on the analytic solution (15). The collocation points are sampled from 41 points for the boundary conditions and 2601 points for the residual points for the governing equation. The solution curve is obtained by specifying Dirichlet boundary conditions from the analytic solution [56]. Several works examining the PINN solution for Kovasznay flow with similar configurations can be found in [38,39]. The proposed elaboration replaces the tanh activation at the last hidden layer with the ADA-F layer, the scale of half-range expansion $L = 1$, the number of orthogonal modes $N_m = 3$ and the number of linear pieces $N_P = 3$. The identity linear mapping is employed in the ADA-F layer. The neural networks prompt u, v and p given the spatial input of x and y, as schematically illustrated in Fig. 6(b).

Figs. 6(b) and (c) compares the capability of the baseline PINN (blue solid line) and ADA-F extension for PINN (red solid line). The $L2$ relative error of velocity v is reported as a performance evaluation. The ADA-F extension presents better accuracy and convergency compared to the vanilla PINN. Contours of velocity and pressure field as shown in Figures 7(d)-(f) confirm that the ADA-F extension

Table 3
Static Euler Beam: Comparison of capabilities between the baseline and ADA-F extension across varying neural network architectures. Metrics used include training loss, $L1$ absolute errors, and $L2$ relative errors. Text in bold black denotes the best-performing result. The top results are presented from ten trials, based on parallel computations.

Architectures		Baseline			ADA-F		
Layers	Neurons	Loss	$L1$ absolute	$L2$ relative	Loss	$L1$ absolute	$L2$ relative
2	10	2.428E-09	4.772E-05	9.701E-04	1.997E-015	1.650E-08	3.186E-07
2	20	4.294E-08	1.327E-05	2.377E-04	**1.424E-015**	**1.489E-08**	**2.542E-07**
4	10	4.627E-09	2.110E-06	4.283E-05	6.887E-016	2.568E-08	4.453E-07
4	20	5.143E-08	1.885E-05	3.930E-04	9.836E-016	1.833E-08	3.646E-07

Table 4
Static Euler Beam: Comparison of Convergence Between the Baseline and ADA-F Extension Across Varying Neural Network Architectures. Metrics used include the number of iterations, CPU time in seconds per epoch, and total elapsed time. The convergence results are based on the best-performing instances, determined by training loss, from five repetitive runs.

Architectures		Baseline			ADA-F		
Layers	Neurons	No. of Iterations	Time/Epoch (*sec*/epoch)	Time (*sec*)	No. of Iterations	Time/Epoch (*sec*/epoch)	Time (*sec*)
2	10	956	0.0034	3.249	216	0.0169	3.640
2	20	535	0.0041	2.218	220	0.0165	3.640
4	10	539	0.0068	3.655	215	0.0182	3.901
4	20	539	0.0073	3.952	221	0.0184	4.077

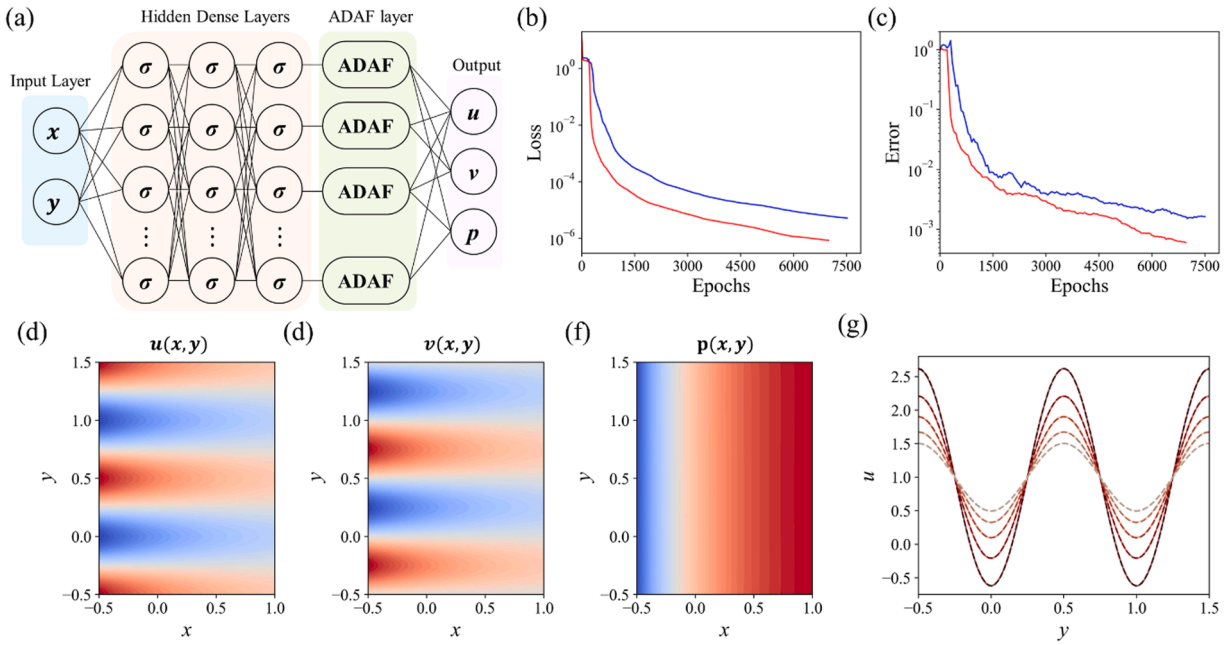


Fig. 6. Comparison of Vanilla PINN and ADA-F Extension for the Kovasznay Flow. (a) Schematic illustration of the ADA-F extension. (b) Plots of training loss and (c) $L2$ relative error versus epochs obtained from neural networks with 2 hidden layers of 60 neurons each, where the blue and red lines represent the results of the baseline model and the ADA-F extension, respectively. The predictions of the ADA-F extension are displayed in contour plots for (d) velocity component u, (e) velocity component v, and (f) pressure p. Additionally, (g) presents line plots of velocity component u versus y, with different shades of red indicating various positions of constant x.

accurately capture the alternating flow fields in special domain of the Kovasznay flow. The comparison of exact solution and prediction of ADA-F extension for x-directional velocity is exhibited in Figure 7(g), where the dark to light red colors indicate different position of x which ranges from -0.5 to 0.71. The inertia of fluid flow decays by advancing direction balancing the viscous stress force, as also presented in the pressure field in (f).

Table 5 compares the capabilities of the baseline PINN and the ADA-F architecture by varying the neural network architecture. The metrics used for comparison include training loss, $L1$ absolute errors, and $L2$ relative errors for the velocity component v. Accurately

Table 5
Kovasznay flow (Re=40): Comparison of Capabilities Between the Baseline and ADA-F Extension Across Varying Neural Network Architectures. Metrics used include training loss, $L1$ absolute errors, and $L2$ relative errors. Text in bold black denotes the best-performing result. The top results are presented from ten trials, based on parallel computations.

Architectures		Baseline			ADA-F		
Layers	Neurons	Loss	$L1$ absolute	$L2$ relative	Loss	$L1$ absolute	$L2$ relative
2	20	1.224E-05	1.917E-04	2.855E-03	1.584E-06	7.223E-05	9.744E-04
2	40	9.728E-06	1.653E-04	2.250E-03	1.235E-06	7.061E-05	9.166E-04
2	60	5.205E-06	1.041E-04	1.633E-03	8.582E-07	3.987E-05	5.977E-04
4	20	7.257E-06	1.517E-04	2.372E-03	4.948E-06	1.132E-04	1.549E-03
4	40	1.847E-06	9.105E-05	1.248E-03	1.155E-06	7.091E-05	1.142E-03
4	60	1.618E-06	7.184E-05	1.056E-03	**4.124E-07**	**3.954E-05**	**5.380E-04**

capturing the y-directional velocity, which emanates from vorticity diffusion over the shear layer, is a crucial factor in assessing the quality of the proposed solution. It is noteworthy that the ADA-F extension can achieve a four-digit decimal relative error with an extremely shallow neural network consisting of 2 hidden layers of 20 neurons each. This network has only 547 trainable parameters, comprising 543 parameters from the baseline architecture and 4 additional parameters in the ADA-F layer. Table 6 presents the overall accuracy of the velocities u and v, and the pressure field p. The ADA-F extension exhibits slightly better performance compared to the baseline PINN, which has 4 hidden layers of 60 neurons each, corresponding to 11,343 trainable parameters. Thus, the ADA-F extension can reduce the total computation time by approximately fivefold (from 1174 s to 251 s) while using only 1/20 of the trainable parameters to achieve comparable capabilities to the baseline PINN.

Next, the Kovasznay flow at high Reynolds numbers is further examined, as summarized in Table 7. Fluid dynamic problems involving high Reynolds numbers represent one of the significant challenges in PINNs [13]. Previous studies have considered the Kovasznay flow under Reynolds numbers below 100 [37,38]. This study has examined the PINN solution for a Reynolds number of 10, 000. The assumption of steady-state flow necessitates the attached flow under the influence of two-dimensional grids, as qualitatively described in Figures 7(d-f). However, increased inertial effects suppress the development of the shear layer within a narrow range, making it difficult to accurately capture the y-directional velocity v, as evidenced by the large $L2$ relative error in velocity v ranging from 5% to 14% when varying the neural network's architecture. The ADA-F extension can effectively address the limitations of conventional PINNs in resolving high Reynolds number flows. The $L2$ relative errors in the x-directional velocity and pressure field can be improved by up to an order of magnitude. Furthermore, the error in the y-directional velocity is significantly reduced, with about 2.6% errors in the v-directional velocity. These experiments demonstrate remarkable results, showing that the ADA-F layer can be effectively employed to resolve extreme problems, including high Reynolds number flows.

The systematic experiments presented provide a balanced comparison between the baseline PINN and the ADA-F extension, focusing on both efficiency and precision. Table 8 summarizes the experimental results for three example problems: Burger's equation, the static Euler beam problem, and Kovasznay flow. These results examine the number of trainable parameters and the total time budget required to obtain a solution of comparable accuracy, ensuring a fair comparison. The ADA-F extension exhibits outstanding capabilities compared to the baseline PINN, enabling the achievement of comparable or superior solutions with significantly fewer trainable parameters and lower time consumption, corresponding to three to fivefold reductions in both parameters and time.

5. Conclusion

This study has proposed a novel method to construct a univariate approximator based on the anti-derivatives of the Fourier series expansion, termed as ADA-F, motivated by the reverse process that begins with a presumed piecewise-linear approximation. The systematic experiments exhibit the outstanding capability of ADA-F in both standalone implementation and incorporation with deep neural networks. The remarkable features of ADA-F, which capture the desired functional relation of a given dataset as well as its anti-derivatives, enable the enhancement of the convergence of deep neural networks and the description of a solution curve for differential equations. Furthermore, ADA-F can be utilized to find the optimal shape of activation, providing a novel method to elaborate neural networks based on unconventional activation formulas specialized for a given dataset.

One significant challenge of ADA-F is that the explicit mathematical formulation could limit flexibility in implementation. The formulation of ADA-F accompanies the series solution, interpreted as for-loops or while-loops in programming languages. This could be a source of significant increase in computational costs and time. Therefore, algorithms to convert the basic ADA-F formulation into vector and matrix operations must be further examined to increase practical usability. The formulation of ADA-F seems to have excessive constraints in terms of training neural networks. The neural networks have a capability to find the optimal solution with reduced constraints. For example, trigonometric formulas and identities can be tested to reduce the complexity of ADA-F formulation without reducing capability. Other orthogonal functions, such as Legendre or Bessel functions, can be further tested.

Although some practical issues and implementation details require further investigation, ADA-F emerges as a novel approximation method with promising capabilities. Notably, it can be seamlessly integrated into PINN to enhance the capabilities of neural network models or to guide architectural design. A noteworthy aspect of the ADA-F extension is its simplicity, achieved by adding just one ADA-F layer with around 10 trainable parameters. Additionally, the ADA-F layer can be integrated with current advancements in PINN techniques, such as enhanced gradients [21], decomposition [22], and parallel implementation [23]. Consequently, ADA-F is expected to serve as a promising tool for solving differential equations, both as a standalone implementation and in conjunction with PINN.

CRediT authorship contribution statement

Jeongsu Lee: Conceptualization, Data curation, Formal analysis, Funding acquisition, Investigation, Methodology, Project administration, Resources, Software, Supervision, Validation, Visualization, Writing – original draft, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Table 6
Kovasznay flow (Re=40): Capability Comparison Between the Baseline and ADA-F Extension for Neural Network Architectures (2 × 20) and (4 × 60). Metrics include $L2$ relative errors (presented as percentages) and time elapsed to convergence.

Baseline (2 × 20)				ADA-F (2 × 20)			
ε_u(%)	ε_v(%)	ε_p(%)	Time (*sec*)	ε_u(%)	ε_v(%)	ε_p(%)	Time (*sec*)
0.3220	0.2855	0.0700	69.155	0.0137	0.0974	0.0287	251.721
Baseline (4 × 60)				**ADA-F** (4 × 60)			
ε_u(%)	ε_v(%)	ε_p(%)	Time (*sec*)	ε_u(%)	ε_v(%)	ε_p(%)	Time (*sec*)
0.0097	0.1056	0.0279	1174.796	0.0072	0.0538	0.0139	4089.366

Table 7
Kovasznay Flow (Re = 10,000): Comparison of Capabilities Between the Baseline and ADA-F Extension Across Varying Neural Network Architectures. Metrics used include training loss, $L1$ absolute errors, and $L2$ relative errors. Text in bold black denotes the best-performing result. The top results are presented from ten trials, based on parallel computations.

Architectures (Layers × Neurons)	Loss	ε_u(%)	ε_v(%)	ε_p(%)
Baseline (2 × 20)	1.002E-06	0.0292	14.991	3.202
Baseline (2 × 40)	7.826E-07	0.0234	11.153	1.369
Baseline (2 × 60)	4.748E-07	0.0187	6.459	1.751
Baseline (4 × 20)	2.919E-07	0.0221	6.996	1.992
Baseline (4 × 40)	4.663E-07	0.0138	8.565	1.492
Baseline (4 × 60)	1.787E-07	0.0252	5.043	1.268
ADA-F (2 × 40)	**3.702E-08**	**0.0073**	**2.633**	**0.489**

Table 8
Summary: Comparison of Vanilla PINN and ADA-F Extension in Terms of the Number of Trainable Parameters and Total Time Budget for Obtaining a Solution with Comparable Accuracy.

Burger's equation				
	$L1$ absolute error	$L2$ relative error	No. of Params.	Time (*sec*)
PINN (4 × 60)	2.518E-04	6.269E-04	11,221	1103.349
ADA-F (4 × 20)	1.774E-04	6.787E-04	1345	356.947
Static Euler beam				
	$L1$ absolute error	$L2$ relative error	No. of Params.	Time (*sec*)
PINN (4 × 10)	2.110E-06	4.283E-05	361	3.655
ADA-F (2 × 10)	1.489E-08	2.542E-07	148	3.640
Kovasznay flow				
	ε_v(%)	ε_p(%)	No. of Params.	Time (*sec*)
PINN (4 × 60)	0.1056	0.0279	11,343	1174.796
ADA-F (2 × 20)	0.0974	0.0287	547	251.721

Data availability

The source codes are available at https://github.com/JeongsLee/

Acknowledgment

This work was supported by the National Research Foundation of Korea (NRF), funded by the Korean government (MSIT), under Grant No. RS-2023-00279929. It was also supported by the Global Industrial Technology Cooperation Program, funded by the Korea Institute for Advancement of Technology (KIAT), under Grant No. P0026191.

Appendix A. Describing location of pieces as trainable variables

The x_i^* can be determined from the optimization by formulating x_i^* with a cumulative summation of positive differences as

$$x_i^* = \sum_{k=0}^{i} \Delta x_k^* \tag{A.1}$$

where $\Delta x_k^* = x_k^* - x_{k-1}^* > 0$ for $k > 0$ with being $\Delta x_0^* = 0$ and $x_0^* = 0$. The soft constraint of the maximum boundary of x_i^* is then added as

$$\underset{y_0,\ \tilde{\lambda}_i,\ x_i^*}{\operatorname{argmin}} \sum \left(f\left(\mathscr{L}^{-1}(x^*)\right) - \tilde{g}(x^*)\right)^2 + \alpha \times \ \operatorname{maximum}\left(0, x_i^* - \mathrm{L}\right)^2 \tag{A.2}$$

where α corresponds to the regularization coefficient.

Appendix B. Formulations of high-order ADA-F

$$\mathrm{ADAF}^0(x^*) = \sum_{m=1}^{N_m}\left[\alpha\frac{LA_m}{m\pi}\sin\frac{m\pi x^*}{L} - (1-\alpha)\frac{LB_m}{m\pi}\left(\cos\frac{m\pi x^*}{L} - 1\right)\right] + \alpha A_0 x^* + y_0 \tag{B.1}$$

$$\begin{aligned}\mathrm{ADAF}^1(x^*) &= \int_0^{x^*}\mathrm{ADAF}^0(v)dv \\ &= -\sum_{m=1}^{N_m}\left[\left(\frac{L}{m\pi}\right)^2\left(\alpha A_m\cos\frac{m\pi x^*}{L} + (1-\alpha)B_m\sin\frac{m\pi x^*}{L}\right)\right] + \frac{\alpha}{2}A_0 x^{*2} + \left[y_0 + (1-\alpha)\sum_{m=1}^{N_m}\left(\frac{LB_m}{m\pi}\right)\right]x^* + \alpha\sum_{m=1}^{N_m}\left[\left(\frac{L}{m\pi}\right)^2 A_m\right] \\ &\quad + y_1\end{aligned}$$

$$\begin{aligned}ADAF^2(x^*) &= \int_0^{x^*}ADAF^1(v)dv \\ &= -\sum_{m=1}^{N_m}\left[\left(\frac{L}{m\pi}\right)^3\left(\alpha A_m\cos\frac{m\pi x^*}{L} - (1-\alpha)B_m\sin\frac{m\pi x^*}{L}\right)\right] + \frac{\alpha}{6}A_0 x^{*3} + \frac{1}{2}\left[y_0 + (1-\alpha)\sum_{m=1}^{N_m}\left(\frac{LB_m}{m\pi}\right)\right]x^{*2} \\ &\quad + \left\{\alpha\sum_{m=1}^{N_m}\left[\left(\frac{L}{m\pi}\right)^2 A_m\right] + y_1\right\}x^* - (1-\alpha)\sum_{m=1}^{N_m}\left[\left(\frac{L}{m\pi}\right)^3 B_m\right] + y_2\end{aligned}$$

$$\begin{aligned}ADAF^3(x^*) &= \int_0^{x^*}ADAF^2(v)dv \\ &= \sum_{m=1}^{N_m}\left[\left(\frac{L}{m\pi}\right)^4\left(\alpha A_m\cos\frac{m\pi x^*}{L} + (1-\alpha)B_m\sin\frac{m\pi x^*}{L}\right)\right] + \frac{\alpha}{24}A_0 x^{*4} + \frac{1}{6}\left[y_0 + (1-\alpha)\sum_{m=1}^{N_m}\left(\frac{LB_m}{m\pi}\right)\right]x^{*3} \\ &\quad + \frac{1}{2}\left\{\alpha\sum_{m=1}^{N_m}\left[\left(\frac{L}{m\pi}\right)^2 A_m\right] + y_1\right\}x^{*2} + \left\{-(1-\alpha)\sum_{m=1}^{N_m}\left[\left(\frac{L}{m\pi}\right)^3 B_m\right] + y_2\right\}x^* - \alpha\sum_{m=1}^{N_m}\left[\left(\frac{L}{m\pi}\right)^4 A_m\right] + y_3\end{aligned}$$

Appendix C. Baseline configuration

The optimal baseline configuration for the PINN is investigated by varying activation functions and optimizers as follows: The Sigmoid and hyperbolic tangent (tanh) activation functions are examined, while the optimizers Adam [54], AdamW [55], and -BFGS-B [46] are tested. This experiment solves Burger's equation using feed-forward neural networks with 4 layers, each containing 20 neurons. Trends of loss and accuracy over 4000 training epochs are presented in Fig. C.1. Figures C.1(a, b) correspond to the Sigmoid activation, and (c, d) correspond to the tanh activation. The black, blue, and red lines represent the results of Adam, AdamW, and L-BFGS-B, respectively. The experiments demonstrate that L-BFGS-B combined with the tanh activation exhibits the most superior performance in terms of training loss and accuracy. Consequently, the baseline PINN configuration considered in this study utilizes the tanh activation and L-BFGS-B optimizer. The Adam optimizer is employed to initiate training for the first 200 epochs. The adopted configuration for the baseline PINN aligns with previous publications [1].

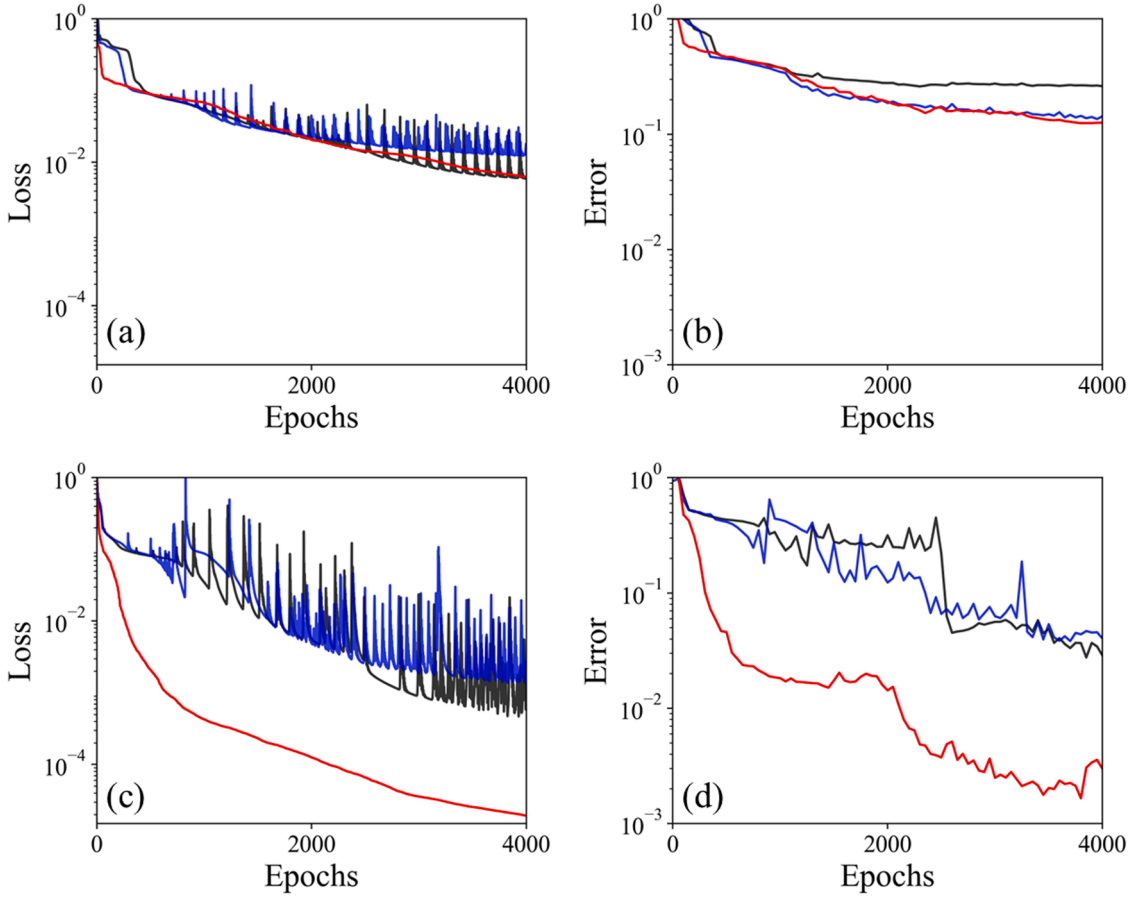


Fig. C.1. Training loss and $L2$ relative error over epochs for solving Burger's equation using baseline PINNs with different activation functions: (a, b) Sigmoid and (c, d) tanh. The black, blue, and red lines represent the results from the Adam, AdamW, and L-BFGS-B optimizers, respectively.

Appendix D. Configuration of Hyperparameters for ADA-F

The naive formulation of ADA-F involves various hyperparameters, which are summarized in Table D.1. The optimal configuration, obtained by varying these hyperparameters, is examined to guide the implementation of ADA-F. The Kovasznay flows, with Reynolds numbers of 10,000, serve as an example case to assess the capabilities of the ADA-F extension. The neural network architecture, featuring 2 hidden layers with 20 neurons each, is utilized as a test configuration. The experiments were conducted over five repetitive runs, and the best results are presented.

Table D.1
Definitions and suggested configurations of hyperparameters for ADA-F.

Hyperparameters	Definition	Suggested Configuration
N_p	Number of linear pieces	Adjustable for standalone; ≥ 3 for bottom layer in PINN; ≥ 15 for top layer in PINN
N_m	Number of orthogonal modes	Equal to N_p
L	Mapped scale	Order(1)
γ	Mapped scale factor	1.0 for PINN; 0.2 for system of ODEs solver in standalone
α	Balancing factor between odd and even expansions	1.0

First, the effectiveness of the mathematical configuration for using both even and odd functions, such as sine and cosine expansions, is examined based on Equation (D.1):

$$\frac{d\widetilde{g}(x^*)}{dx^*} = \sum_{m=0}^{\infty}\left(\alpha A_m \cos\frac{m\pi x^*}{L} + (1-\alpha)B_m \sin\frac{m\pi x^*}{L}\right) \tag{D.1}$$

This formula is equivalent to Eq. (1), where the coefficient α adjusts the relative influence of odd and even functions. The conditions for balancing functions, corresponding to $\alpha = 1.0$ and $\alpha = 0.5$, are examined and summarized in Table D.2. The experiments show that using the Fourier cosine expansion alone, corresponding to $\alpha = 1.0$, demonstrates slightly better capability compared to using both cosine and sine expansions. Therefore, it is an efficient configuration to use only the Fourier cosine or sine expansion to reduce additional gradient computations. The choice between odd and even expansions could be considered based on the expected solution profiles.

Table D.2
Kovasznay Flow ($Re = 10{,}000$): Capability Comparison – This table compares the effects of varying the balancing factor (α) between odd and even expansions, set at 0.5 and 1.0, respectively. Metrics included are training loss and L2 relative errors, presented as percentages.

α	Loss	ε_u(%)	ε_v(%)	ε_p(%)
0.5	4.077E-07	0.0153	8.942	2.279
1.0	1.036E-07	0.0074	7.064	1.206

Next, the effect of the length L is examined by selecting values from the set [0.5, 1.0, 2.0, 5.0, 10.0], and the results are summarized in Table D3. The mapping scale L did not significantly influence the capability of the ADA-F solution, which is consistent with expectations, as neural networks are designed to rescale hidden variables through the Dense layer. However, the capability of the ADA-F significantly diminishes when L increases to 10.0. It is crucial to balance the hidden variables and the scale of the weights to facilitate the training process. For example, normalizing the input variables significantly impacts the neural network's performance. Therefore, adjusting the scale of the mapping dimension within the range of 0.5 to 5.0 is recommended.

Table D.3
Kovasznay Flow ($Re = 10{,}000$): Capability Comparison – This table illustrates the effects of varying L from 0.5 to 10. Metrics evaluated include training loss and L2 relative errors, which are presented as percentages.

L	Loss	ε_u(%)	ε_v(%)	ε_p(%)
0.5	1.705E-07	0.0197	5.955	1.432
1	1.036E-07	0.0074	7.064	1.206
2	5.236E-07	0.0498	8.638	2.039
5	2.226E-07	0.0167	6.033	1.049
10	1.447E-06	0.0404	8.767	2.516

The number of linear pieces and truncated modes are the most important hyperparameters, as they determine the expression capability of the ADA-F layer as well as the computational costs. Experiments varying the number of linear pieces and truncation modes were conducted, as summarized in Table D.4. Overall, the capability of the ADA-F extension gradually increases with the number of linear pieces; the highest number of linear pieces and modes, corresponding to nine, exhibits the best performance. However, the capability of an ADA-F extension with more than three linear pieces and modes shows comparable performance to that of an extension with nine modes. Meanwhile, this study employs the same number of linear modes and pieces to ensure that the coefficients of the modes and the slope of the panels are identical, thereby ensuring that the model has the same number of dimensional freedoms as the desired piecewise linear approximation. However, this is not a necessary requirement, as the ADA-F extension with different numbers of modes and pieces also performs well, as shown in Table D4.

Table D.4
Kovasznay Flow ($Re = 10{,}000$): Capability Comparison – This table illustrates the effects of varying the number of linear pieces (N_P) and the number of orthogonal modes (N_m). Metrics evaluated include training loss and L2 relative errors, presented as percentages.

	N_m	Loss	ε_u(%)	ε_v(%)	ε_p(%)
3	3	1.472E-07	0.0131	6.130	1.120
5	3	1.052E-07	0.0094	7.485	1.420
5	5	1.140E-07	0.0114	7.480	1.226
5	7	1.115E-07	0.0098	5.583	0.857
7	7	6.905E-08	0.0111	5.544	0.708
9	9	6.678E-08	00,092	5.074	0.944

Lastly, the effectiveness of neural network architectures incorporating multiple ADA-F layers is examined. The architecture is modified to include two ADA-F layers: one at the top, immediately following the input variables, and another at the final layer. The top ADA-F layer is configured with 25 pieces following the linear Dense layer, enabling the linear combination of inputs (x and y in Kovasznay flow problems) to be mapped to a non-linear relationship via ADA-F. The last layer is configured with three pieces and modes, consistent with configurations used in other experiments. Experimental results are presented in Table D5. The experiments

show that by employing an ADA-F layer at the top, the capability of the neural networks can be significantly improved, showing almost an order-of-magnitude improvement compared to a single-layer ADA-F configuration. However, the input layer ADA-F is more effective with a higher number of pieces, above 15. This increase can lead to longer computational times, thereby creating an inefficient configuration for certain problems. Although the dual-ADA-F (D-ADA-F) configuration may increase computational time, it helps to address extreme problems using simple neural network architectures, as tested in this study with high Reynolds number problems of 10,000. Thus, it has significant potential to be employed to overcome the limitations of PINN.

Table D.5
Kovasznay Flow ($Re = 10{,}000$): Capability comparison of ADA-F and D-ADA-F.

Architecture	Loss	ε_u(%)	ε_v(%)	ε_p(%)
ADAF	1.140E-07	0.0114	7.480	1.226
D-ADAF	3.902E-08	0.0034	2.329	0.404

References

[1] M. Raissi, P. Perdikaris, G.E. Karniadakis, Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, J. Comput. Phys. 378 (2019) 686–707.
[2] G.E. Karniadakis, I.G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, L. Yang, Physics-informed machine learning, Nat. Rev. Phys. 3 (2021) 422–440.
[3] S. Cuomo, V.S. Di Cola, F. Giampaolo, G. Rozza, M. Raissi, F. Piccialli, Scientific machine learning through physics–informed neural networks: where we are and what's next, J. Sci. Comput. 92 (3) (2022) 88.
[4] S. Cai, Z. Wang, S. Wang, P. Perdikaris, G.E. Karniadakis, Physics-informed neural networks for heat transfer problems, J. Heat Transfer 143 (6) (2021) 060801.
[5] Z. Mao, A.D. Jagtap, G.E. Karniadakis, Physics-informed neural networks for high-speed flows, Comput. Methods Appl. Mech. Engrg. 360 (2020) 112789.
[6] S. Alkhadhr, X. Liu, M. Almekkawy, Modeling of the forward wave propagation using physics-informed neural networks, in: 2021 IEEE International Ultrasonics Symposium (IUS), IEEE, 2021, pp. 1–4.
[7] Y. Chen, L. Lu, G.E. Karniadakis, L. Dal Negro, Physics-informed neural networks for inverse problems in nano-optics and metamaterials, Opt. Express 28 (8) (2020) 11618–11633.
[8] A.D. Jagtap, Z. Mao, N. Adams, G.E. Karniadakis, Physics-informed neural networks for inverse problems in supersonic flows, J. Comput. Phys. 466 (2022) 111402.
[9] I. Depina, S. Jain, S. Mar Valsson, H. Gotovac, Application of physics-informed neural networks to inverse problems in unsaturated groundwater flow, Georisk: assess, Manag. Risk Eng. Syst. Geohaz. 16 (1) (2022) 21–36.
[10] E. Kharazmi, D. Fan, Z. Wang, M.S. Triantafyllou, Inferring vortex induced vibrations of flexible cylinders using physics-informed neural networks, J. Fluids Struct. 107 (2021) 103367.
[11] Z. Lai, C. Mylonas, S. Nagarajaiah, E. Chatzi, Structural identification with physics-informed neural ordinary differential equations, J. Sound Vib. 508 (2021) 116196.
[12] X.Y. Guo, S.E. Fang, Structural parameter identification using physics-informed neural networks, Meas 220 (2023) 113334.
[13] S. Cai, Z. Mao, Z. Wang, M. Yin, G.E. Karniadakis, Physics-informed neural networks (PINNs) for fluid mechanics: a review, Acta Mech. Sin. 37 (12) (2021) 1727–1738.
[14] S. Cai, Z. Wang, C. Chryssostomidis, G.E. Karniadakis, Heat transfer prediction with unknown thermal boundary conditions using physics-informed neural networks, Fluids Eng. Div. Summer Meet. 83730 (2020). V003T05A054.
[15] X. Zhao, Z. Gong, Y. Zhang, W. Yao, X. Chen, Physics-informed convolutional neural networks for temperature field prediction of heat source layout without labeled data, Eng. Appl. Artif. Intell. 117 (2023) 105516.
[16] C.A. Yan, R. Vescovini, L. Dozio, A framework based on physics-informed neural networks and extreme learning for the analysis of composite structures, Comput. Struct. 265 (2022) 106761.
[17] Z. Bai, S. Song, Structural reliability analysis based on neural networks with physics-informed training samples, Eng. Appl. Artif. Intell. 126 (2023) 107157.
[18] K. Linka, A. Schäfer, X. Meng, Z. Zou, G.E. Karniadakis, E. Kuhl, Bayesian Physics Informed Neural Networks for real-world nonlinear dynamical systems, Comput. Methods Appl. Mech. Engrg. 402 (2022) 115346.
[19] F. Djeumou, C. Neary, E. Goubault, S. Putot, U. Topcu, Neural networks with physics-informed architectures and constraints for dynamical systems modeling, Learn. Dyn. Control Conf., PMLR (2022) 263–277.
[20] A. Thangamuthu, G. Kumar, S. Bishnoi, R. Bhattoo, N.M. Krishnan, S. Ranu, Unravelling the performance of physics-informed graph neural networks for dynamical systems, Adv. Neural Inf. Process. Syst. 35 (2022) 3691–3702.
[21] J. Yu, L. Lu, X. Meng, G.E. Karniadakis, Gradient-enhanced physics-informed neural networks for forward and inverse PDE problems, Comput. Methods Appl. Mech. Eng. 393 (2022) 114823.
[22] E. Kharazmi, Z. Zhang, G.E. Karniadakis, hp-VPINNs: variational physics-informed neural networks with domain decomposition, Comput. Methods Appl. Mech. Engrg. 374 (2021) 113547.
[23] K. Shukla, A.D. Jagtap, G.E. Karniadakis, Parallel physics-informed neural networks via domain decomposition, J. Comput. Phys. 447 (2021) 110683.
[24] L.D. McClenny, U.M. Braga-Neto, Self-adaptive physics-informed neural networks, J. Comput. Phys. 474 (2023) 111722.
[25] J. Yang, X. Liu, Y. Diao, X. Chen, H. Hu, Adaptive task decomposition physics-informed neural networks, Comput. Methods Appl. Mech. Engrg. 418 (2024) 116561.
[26] D. Dalton, D. Husmeier, H. Gao, Physics-informed graph neural network emulation of soft-tissue mechanics, Comput. Methods Appl. Mech. Engrg. 417 (2023) 116351.
[27] W. Liu, M.J. Pyrcz, Physics-informed graph neural network for spatial-temporal production forecasting, Geoenergy Sci. Eng. 223 (2023) 211486.
[28] J.Z. Peng, N. Aubry, Y.B. Li, M. Mei, Z.H. Chen, W.T. Wu, Physics-informed graph convolutional neural network for modeling geometry-adaptive steady-state natural convection, Int. J. Heat Mass Transf. 216 (2023) 124593.
[29] J.Z. Peng, Y. Hua, Y.B. Li, Z.H. Chen, W.T. Wu, N. Aubry, Physics-informed graph convolutional neural network for modeling fluid flow and heat convection, Phys. Fluids 35 (8) (2023).
[30] S. Hochreiter, The vanishing gradient problem during learning recurrent neural nets and problem solutions, Int. J. Uncertain. Fuzziness Knowl.-Based Syst. 6 (02) (1998) 107–116.
[31] B. Hanin, Which neural net architectures give rise to exploding and vanishing gradients? Adv. Neural Inf. Process. Syst. (2018) 31.
[32] T. Szandała, Review and comparison of commonly used activation functions for deep neural networks, Bio-inspired Neurocomput (2021) 203–224.
[33] S.R. Dubey, S.K. Singh, B.B. Chaudhuri, Activation functions in deep learning: a comprehensive survey and benchmark, Neurocomputing (2022).

[34] A.D. Jagtap, K. Kawaguchi, G.E. Karniadakis, Adaptive activation functions accelerate convergence in deep and physics-informed neural networks, J. Comput. Phys. 404 (2020) 109136.
[35] A.D. Jagtap, E. Kharazmi, G.E. Karniadakis, Conservative physics-informed neural networks on discrete domains for conservation laws: applications to forward and inverse problems, Comput. Methods Appl. Mech. Engrg. 365 (2020) 113028.
[36] A.D. Jagtap, Y. Shin, K. Kawaguchi, G.E. Karniadakis, Locally adaptive activation functions with slope recovery for deep and physics-informed neural networks, Proc. R. Soc. A 476 (2239) (2020) 20200334.
[37] A.D. Jagtap, Y. Shin, K. Kawaguchi, G.E. Karniadakis, Deep Kronecker neural networks: a general framework for neural networks with adaptive activation functions, Neurocomputing 468 (2022) 165–180.
[38] X. Jin, S. Cai, H. Li, G.E. Karniadakis, NSFnets (Navier-Stokes flow nets): physics-informed neural networks for the incompressible Navier-Stokes equations, J. Comput. Phys. 426 (2021) 109951.
[39] Q. Lou, X. Meng, G.E. Karniadakis, Physics-informed neural networks for solving forward and inverse flow problems via the Boltzmann-BGK formulation, J. Comput. Phys. 447 (2021) 110676.
[40] Z. Xiang, W. Peng, X. Liu, W. Yao, Self-adaptive loss balanced physics-informed neural networks, Neurocomputing 496 (2022) 11–34.
[41] E.J.R. Coutinho, M. Dall'Aqua, L. McClenny, M. Zhong, U. Braga-Neto, E. Gildin, Physics-informed neural networks with adaptive localized artificial viscosity, J. Comput. Phys. 489 (2023) 112265.
[42] Y. He, Z. Wang, H. Xiang, X. Jiang, D. Tang, An artificial viscosity augmented physics-informed neural network for incompressible flow, Appl. Math. Mech. 44 (7) (2023) 1101–1110.
[43] J. Choi, T. Yun, N. Kim, Y. Hong, Spectral operator learning for parametric PDEs without data reliance, Comput. Methods Appl. Mech. Engrg. 420 (2024) 116678.
[44] A. Pinkus, Approximation theory of the MLP model in neural networks, Acta Numer 8 (1999) 143–195.
[45] D. Jackson, Fourier Series and Orthogonal Polynomials, Courier Corporation (2004).
[46] C. Zhu, R.H. Byrd, P. Lu, J. Nocedal, Algorithm 778: L-BFGS-B: Fortran subroutines for large-scale bound-constrained optimization, ACM Trans. Math. Softw. 23 (1997) 550–560.
[47] P. Virtanen, R. Gommers, T.E. Oliphant, et al., SciPy 1.0: fundamental algorithms for scientific computing in Python, Nat. Methods 17 (3) (2020) 261–272.
[48] F. de Avila Belbute-Peres, Y.F. Chen, F. Sha, HyperPINN: learning parameterized differential equations with physics-informed hypernetworks, in: The Symbiosis of Deep Learning and Differential Equations, 2021.
[49] S. Wang, S. Sankaran, P. Perdikaris, Preprint at, 2022.
[50] A. Bihlo, R.O. Popovych, Physics-informed neural networks for the shallow-water equations on the sphere, J. Comput. Phys. 456 (2022) 111024.
[51] S. Qian, H. Liu, C. Liu, S. Wu, H.San Wong, Adaptive activation functions in convolutional neural networks, Neurocomputing 272 (2018) 204–212.
[52] M.M. Lau, K.H. Lim, Review of adaptive activation function in deep neural network, in: 2018 IEEE-EMBS Conf. Biomed. Eng. Sci. (IECBES), IEEE, 2018, pp. 686–690.
[53] J.M. Burgers, A mathematical model illustrating the theory of turbulence, Adv. Appl. Mech. 1 (1948) 171–199.
[54] D.P. Kingma, J. Ba, arXiv preprint, 2014.
[55] I. Loshchilov, F. Hutter, arXiv preprint, 2017.
[56] L.I.G. Kovasznay, Laminar flow behind a two-dimensional grid, in: Mathematical Proceeding of the Cambridge Philosophy Society 44, Cambridge University Press, 1948, pp. 58–62.